

STATS 7022 - Data Science

Seminar 12: Fitting Models (Solutions)

Max Glonek

Trimester 1, 2025

Load the Data and Libraries

```
pacman::p_load(tidyverse, tidymodels, discrim, magrittr)
data <- read_rds("data.rds")
```

Regression Models

Linear Regression With Interaction Term

```
# fit the model
lm_1 <- lm(X2 ~ X1*X3, data = data)

# fit with tidymodels
recipe_lm <- recipe(X2 ~ ., data = data) %>%
  step_interact(terms = ~ X1:X3)
recipe_lm %>% prep() %>% bake(new_data = NULL)
```

```
## # A tibble: 400 × 4
##       X1 X3       X2 X1_x_X31
##   <dbl> <fct> <dbl>   <dbl>
## 1 12.2  1      24.0    12.2
## 2 13.7  1      25.3    13.7
## 3 15.9  1      30.9    15.9
## 4  8.60 1      15.8     8.60
## 5  7.95 1      12.1     7.95
## 6 12.0  1      21.9    12.0
## 7  7.40 1      13.9     7.40
## 8  9.55 1      18.6     9.55
## 9 15.3  1      30.2    15.3
## 10 5.79 1      10.2     5.79
## # i 390 more rows
```

```

model_lm <- linear_reg() %>%
  set_engine("lm")

workflow_lm <- workflow() %>%
  add_recipe(recipe_lm) %>%
  add_model(model_lm)

fit_lm <- workflow_lm %>%
  fit(data)

fit_lm %>% tidy()

```

```

## # A tibble: 4 × 5
##   term          estimate std.error statistic  p.value
##   <chr>         <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)   43.9       0.447      98.2 4.65e-280
## 2 X1           -1.51      0.0336    -45.1 5.10e-158
## 3 X31          -41.8      0.690    -60.6 2.21e-202
## 4 X1_x_X31      3.35      0.0527     63.6 4.96e-210

```

```

# model summary
summary(lm_1)

```

```

##
## Call:
## lm(formula = X2 ~ X1 * X3, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.5083 -1.1634 -0.0536  1.1394  6.5022
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  43.88349    0.44701   98.17  <2e-16 ***
## X1           -1.51290    0.03356  -45.08  <2e-16 ***
## X31          -41.77327    0.68951  -60.58  <2e-16 ***
## X1:X31        3.35163    0.05267   63.63  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.082 on 396 degrees of freedom
## Multiple R-squared:  0.9116, Adjusted R-squared:  0.9109
## F-statistic: 1361 on 3 and 396 DF,  p-value: < 2.2e-16

```

```

# tidymodels summary
fit_lm %>% extract_fit_parsnip() %>% pluck("fit") %>% summary()

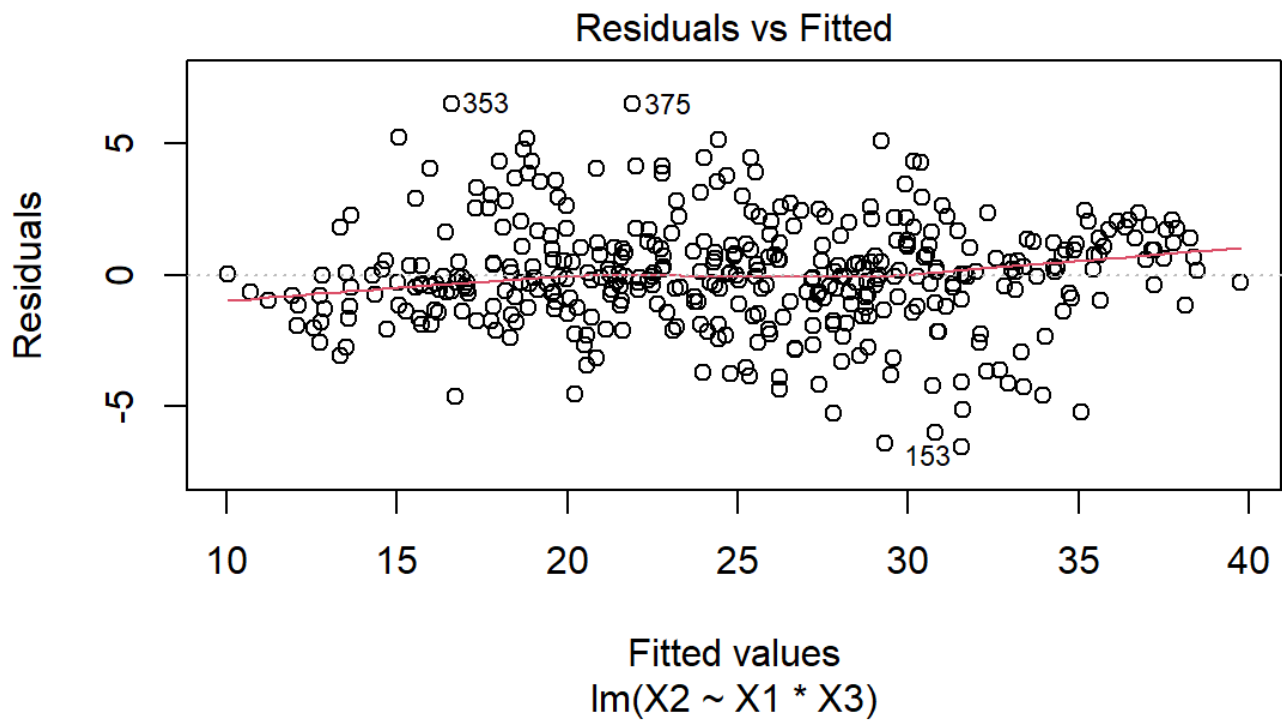
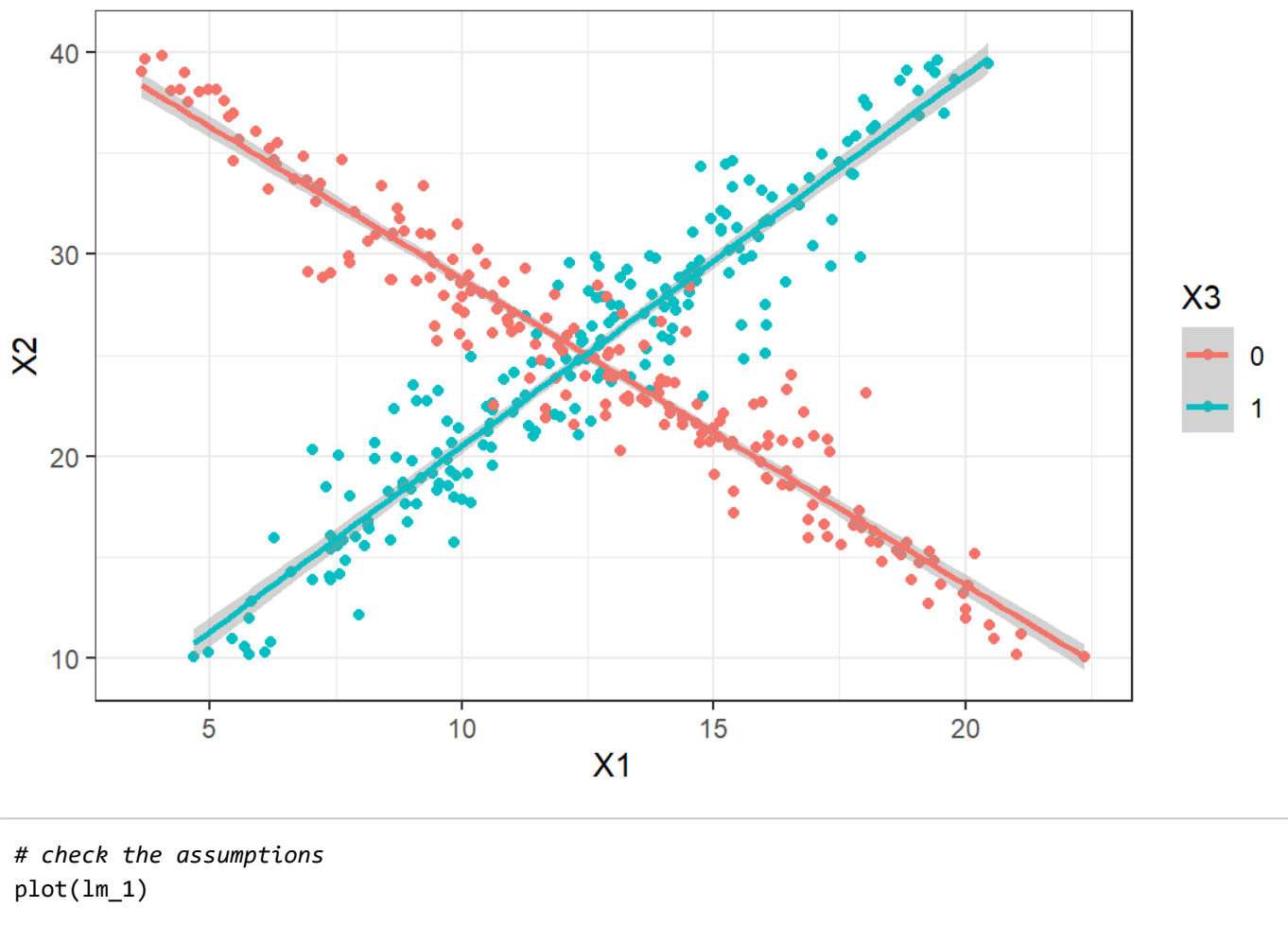
```

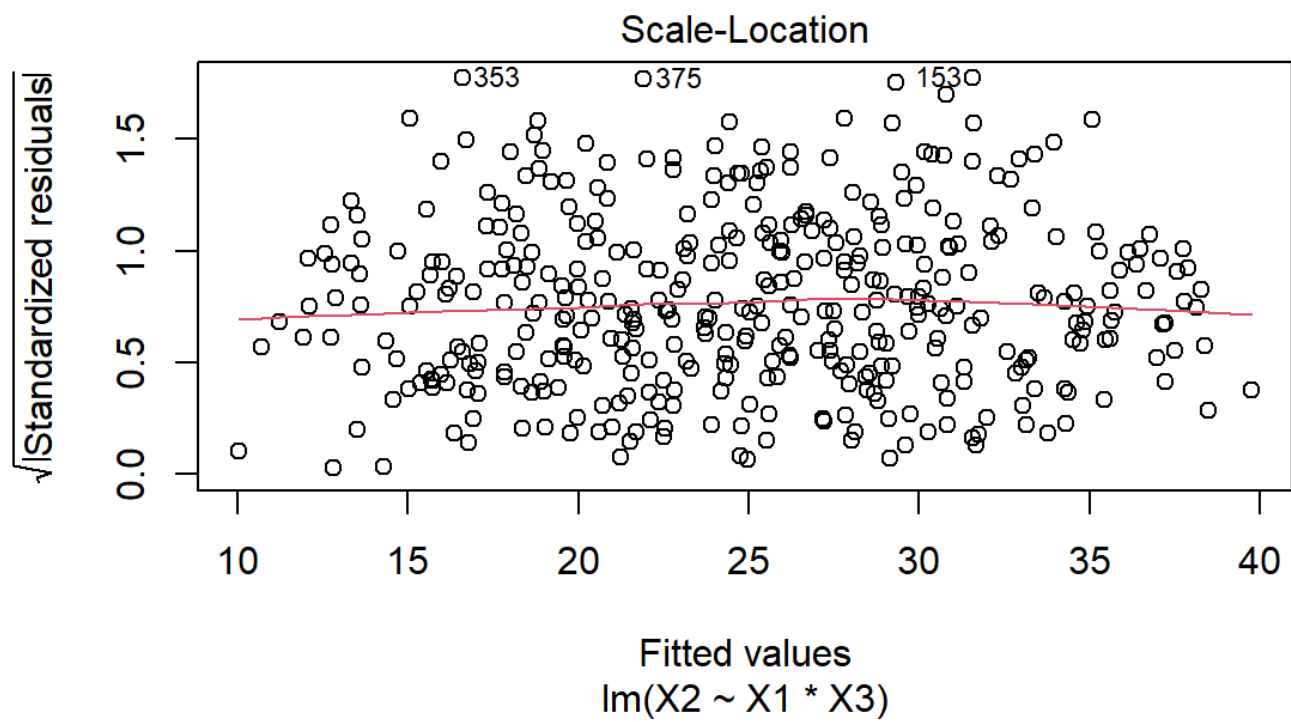
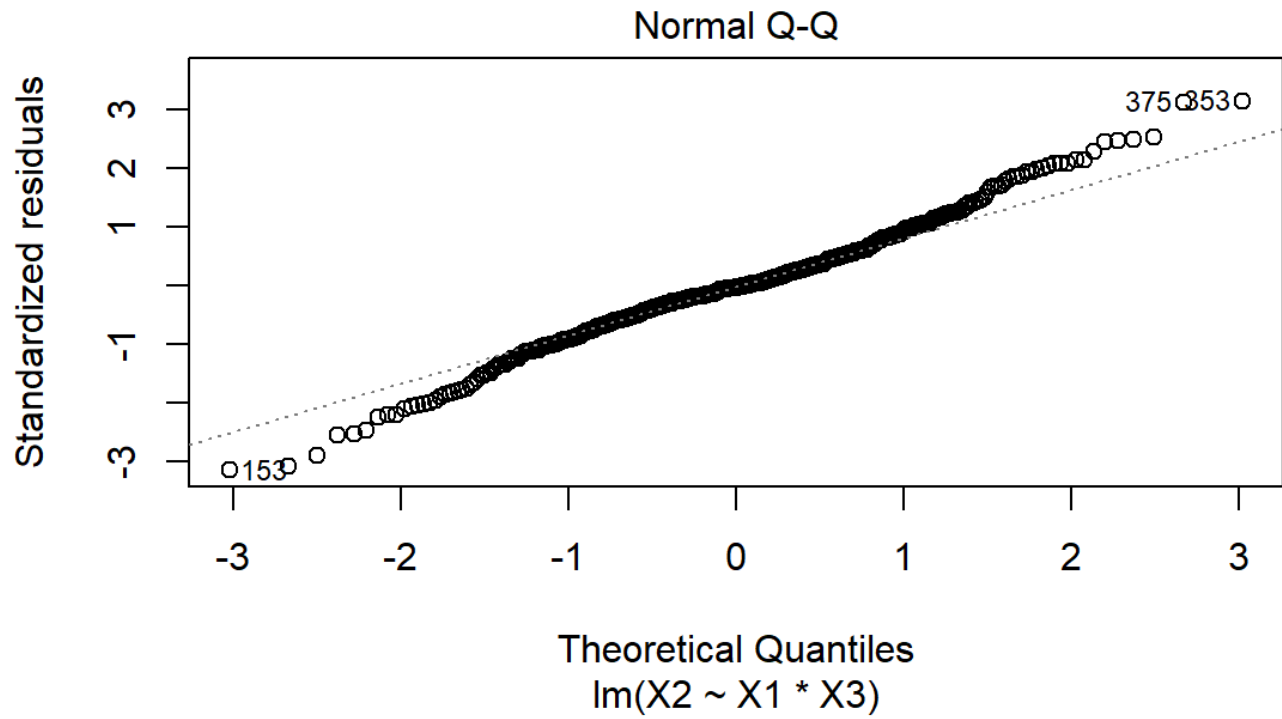
```
##
## Call:
## stats::lm(formula = ..y ~ ., data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.5083 -1.1634 -0.0536  1.1394  6.5022
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  43.88349    0.44701   98.17  <2e-16 ***
## X1           -1.51290    0.03356  -45.08  <2e-16 ***
## X31          -41.77327    0.68951  -60.58  <2e-16 ***
## X1_x_X31      3.35163    0.05267   63.63  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.082 on 396 degrees of freedom
## Multiple R-squared:  0.9116, Adjusted R-squared:  0.9109
## F-statistic: 1361 on 3 and 396 DF,  p-value: < 2.2e-16
```

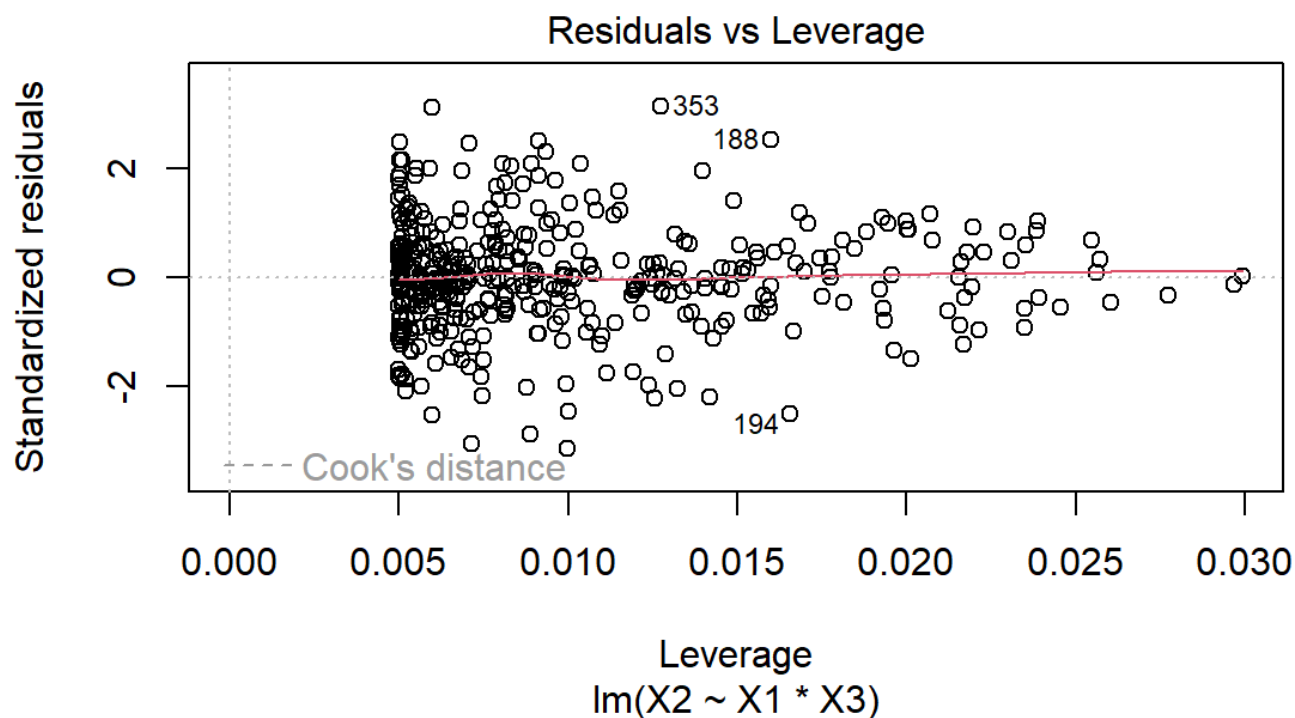
```
metrics <- metric_set(rmse,rsq)
(metrics_lm <- data %>% add_column(predict(fit_lm,data)) %>% metrics(truth = X2, estimate = .
pred))
```

```
## # A tibble: 2 × 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 rmse    standard         2.07
## 2 rsq     standard         0.912
```

```
# visualise the data
data %>%
  ggplot(aes(x = X1, y = X2, colour = X3)) +
  geom_point() +
  geom_smooth(method=lm)
```







Classification Models

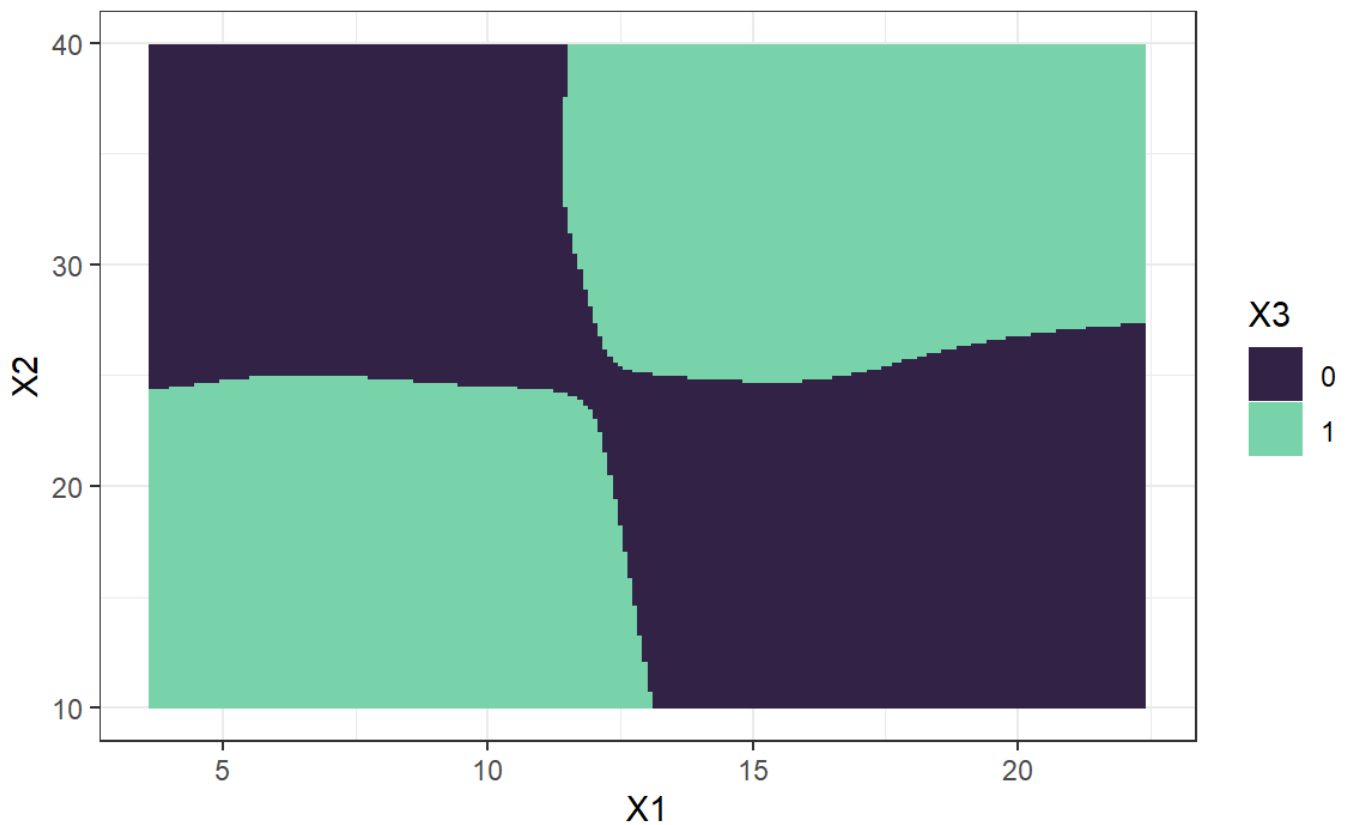
Support Vector Machine With Gaussian Radial Basis Function Kernel

```
# fit the model
svm_r <- recipe(X3 ~., data = data)
svm_m <- svm_rbf() %>%
  set_mode("classification") %>%
  set_engine("kernlab")
svm_wf <- workflow() %>%
  add_recipe(svm_r) %>%
  add_model(svm_m)
svm_fit <- svm_wf %>%
  fit(data)

# model summary
svm_fit %>% extract_fit_parsnip() %>% pluck("fit")
```

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc (classification)
## parameter : cost C = 1
##
## Gaussian Radial Basis kernel function.
## Hyperparameter : sigma = 1.96770046220634
##
## Number of Support Vectors : 125
##
## Objective Function Value : -96.5975
## Training error : 0.0875
## Probability model included.
```

```
# visualise the data
new_data %>%
  add_column(predict(svm_fit, new_data = new_data)) %>%
  ggplot(aes(x=X1,y=X2,fill=.pred_class)) +
  geom_tile() +
  labs(fill = "X3",
       x = "X1",
       y = "X2") +
  theme_bw() +
  viridis::scale_fill_viridis(option="G",
                             begin = 1/6,
                             end = 5/6,
                             discrete=TRUE)
```

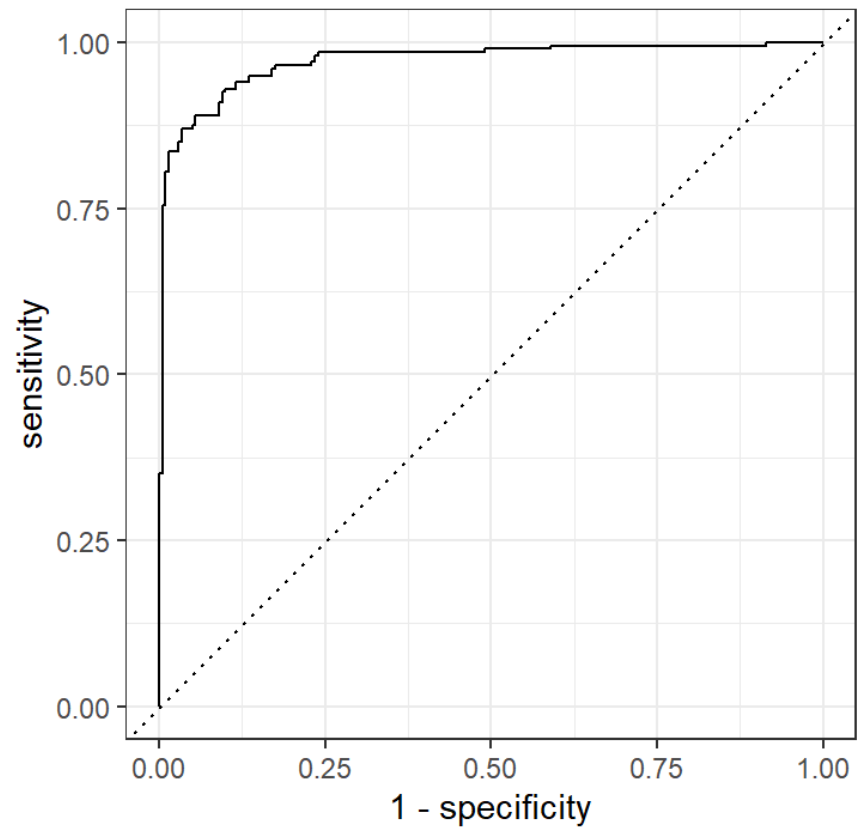


```
# measure performance
svm_pred <- data %>%
  bind_cols(predict(svm_fit, data),
            predict(svm_fit, data, type = "prob"))

svm_pred %>% conf_mat(truth = X3,
                     estimate = .pred_class)
```

```
##           Truth
## Prediction  0   1
##           0 186  21
##           1  14 179
```

```
svm_pred %>% yardstick::roc_curve(truth = X3,
                                  estimate = .pred_0) %>%
  autoplot()
```



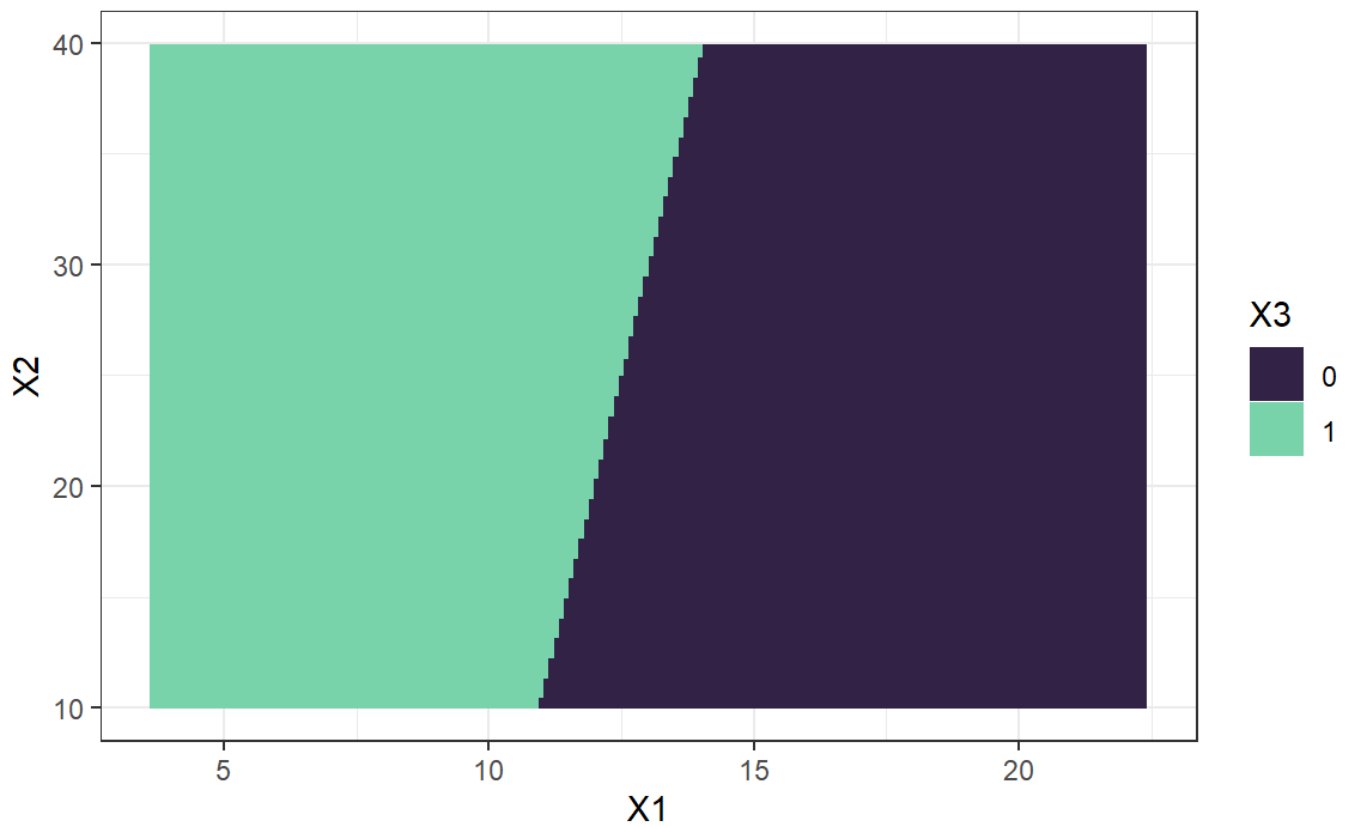
LDA Without Interaction Term

```
# fit the model
lda_r1 <- recipe(X3 ~ ., data = data)
lda_m1 <- discrim_linear() %>%
  set_mode("classification") %>%
  set_engine("MASS")
lda_wf1 <- workflow() %>%
  add_recipe(lda_r1) %>%
  add_model(lda_m1)
lda_fit1 <- lda_wf1 %>%
  fit(data)

# model summary
lda_fit1 %>% extract_fit_parsnip() %>% pluck("fit")
```

```
## Call:
## lda(..y ~ ., data = data)
##
## Prior probabilities of groups:
##   0   1
## 0.5 0.5
##
## Group means:
##           X1           X2
## 0 12.57545 24.85811
## 1 12.41207 24.93275
##
## Coefficients of linear discriminants:
##           LD1
## X1 -0.24024155
## X2  0.02499678
```

```
# visualise the data
new_data %>%
  add_column(predict(lda_fit1, new_data = new_data)) %>%
  ggplot(aes(x=X1,y=X2,fill=.pred_class)) +
  geom_tile() +
  labs(fill = "X3",
       x = "X1",
       y = "X2") +
  theme_bw() +
  viridis::scale_fill_viridis(option="G",
                              begin = 1/6,
                              end = 5/6,
                              discrete=TRUE)
```

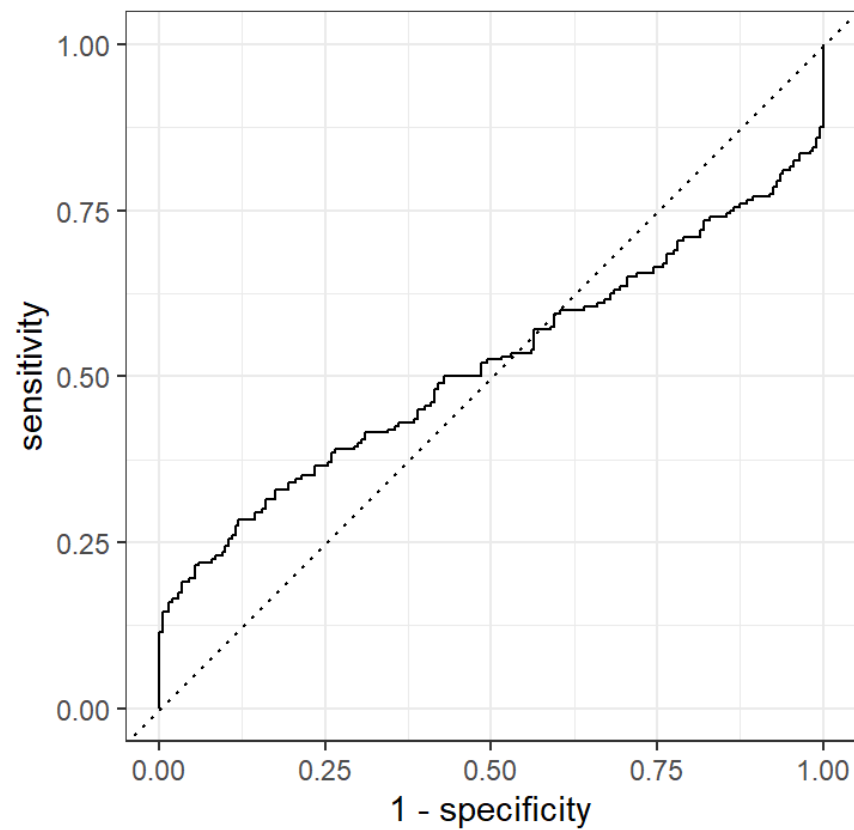


```
# measure performance
lda_pred1 <- data %>%
  bind_cols(predict(lda_fit1, data),
            predict(lda_fit1, data, type = "prob"))

lda_pred1 %>% conf_mat(truth = X3,
                      estimate = .pred_class)
```

```
##           Truth
## Prediction    0    1
##           0 104  98
##           1  96 102
```

```
lda_pred1 %>% yardstick::roc_curve(truth = X3,
                                   estimate = .pred_0) %>%
  autoplot()
```



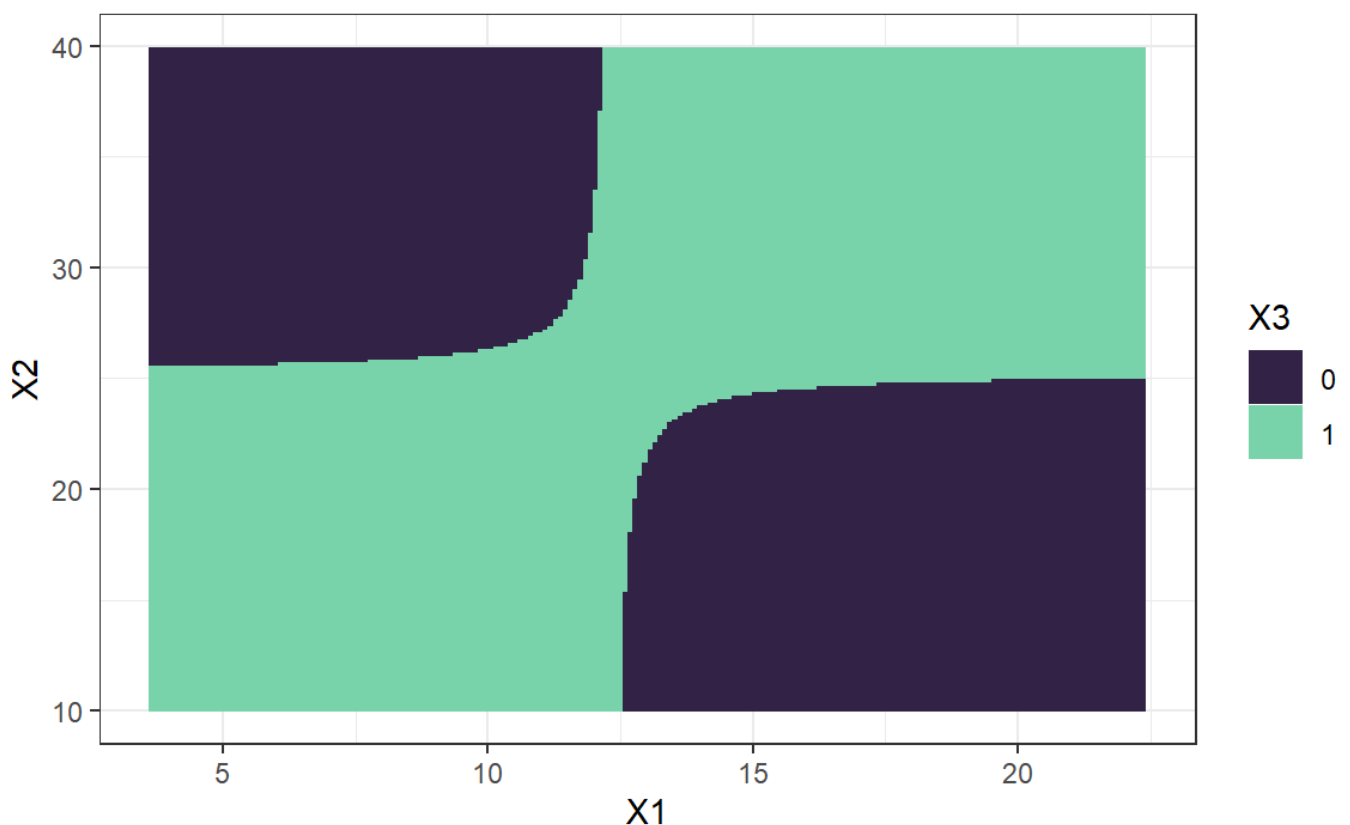
LDA With Interaction Term

```
# fit the model
lda_r2 <- recipe(X3 ~ ., data = data) %>%
  step_interact(terms = ~ X1:X2)
lda_m2 <- discrim_linear() %>%
  set_mode("classification") %>%
  set_engine("MASS")
lda_wf2 <- workflow() %>%
  add_recipe(lda_r2) %>%
  add_model(lda_m2)
lda_fit2 <- lda_wf2 %>%
  fit(data)

# model summary
lda_fit2 %>% extract_fit_parsnip() %>% pluck("fit")
```

```
## Call:
## lda(..y ~ ., data = data)
##
## Prior probabilities of groups:
##    0    1
## 0.5 0.5
##
## Group means:
##      X1      X2  X1_x_X2
## 0 12.57545 24.85811 283.4828
## 1 12.41207 24.93275 333.6542
##
## Coefficients of linear discriminants:
##      LD1
## X1      -0.79045914
## X2      -0.38627908
## X1_x_X2   0.03133188
```

```
# visualise the data
new_data %>%
  add_column(predict(lda_fit2, new_data = new_data)) %>%
  ggplot(aes(x=X1,y=X2,fill=.pred_class)) +
  geom_tile() +
  labs(fill = "X3",
       x = "X1",
       y = "X2") +
  theme_bw() +
  viridis::scale_fill_viridis(option="G",
                              begin = 1/6,
                              end = 5/6,
                              discrete=TRUE)
```

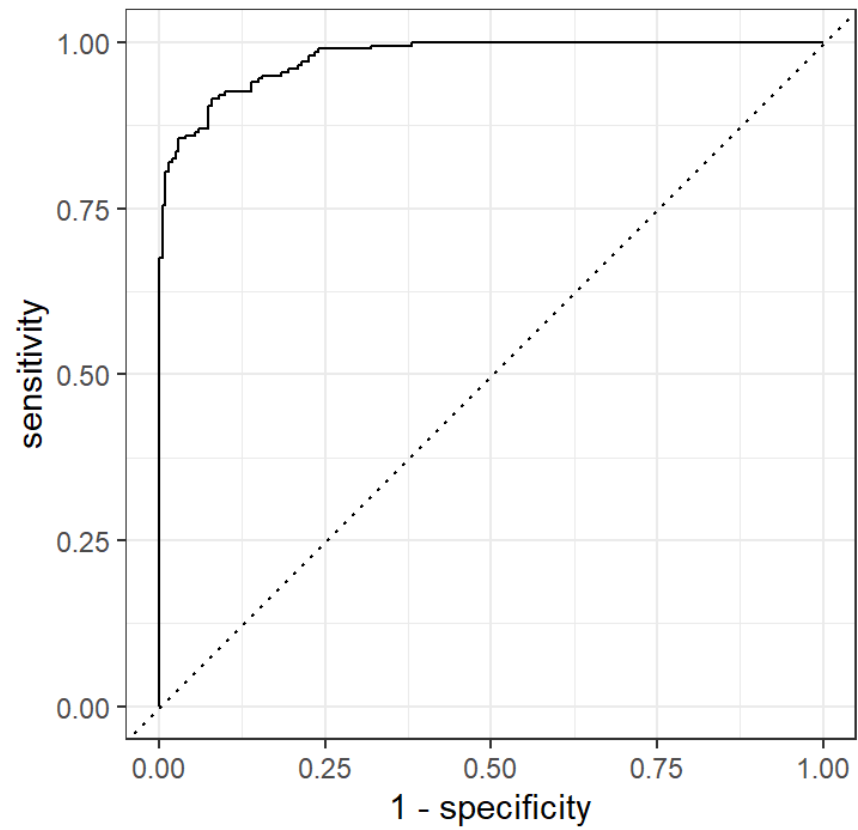


```
# measure performance
lda_pred2 <- data %>%
  bind_cols(predict(lda_fit2, data),
            predict(lda_fit2, data, type = "prob"))

lda_pred2 %>% conf_mat(truth = X3,
                      estimate = .pred_class)
```

```
##           Truth
## Prediction  0   1
##           0 153   2
##           1  47 198
```

```
lda_pred2 %>% yardstick::roc_curve(truth = X3,
                                   estimate = .pred_0) %>%
  autoplot()
```



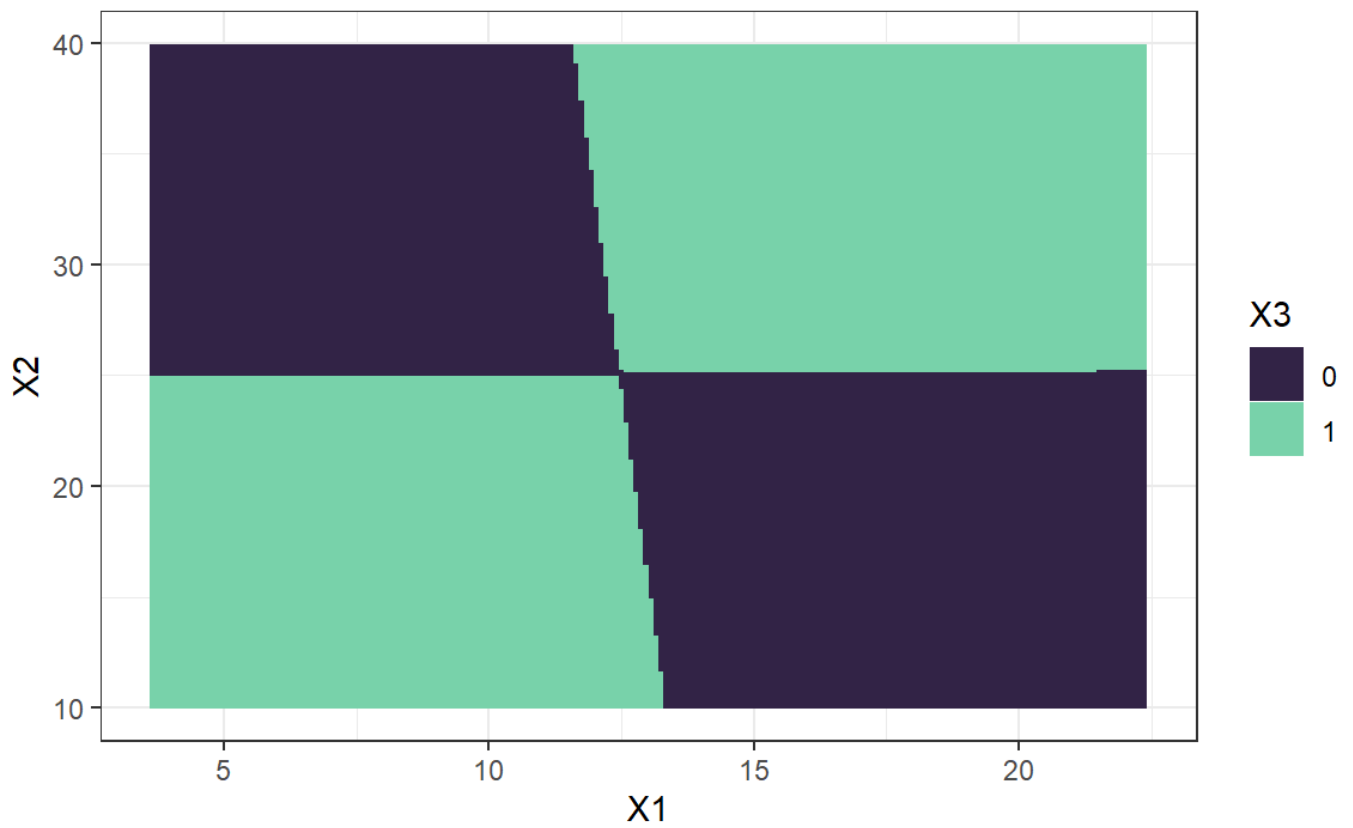
QDA

```
# fit the model
qda_r <- recipe(X3 ~ ., data = data)
qda_m <- discrim_quad() %>%
  set_mode("classification") %>%
  set_engine("MASS")
qda_wf <- workflow() %>%
  add_recipe(qda_r) %>%
  add_model(qda_m)
qda_fit <- qda_wf %>%
  fit(data)

# model summary
qda_fit %>% extract_fit_parsnip() %>% pluck("fit")
```

```
## Call:
## qda(..y ~ ., data = data)
##
## Prior probabilities of groups:
##   0   1
## 0.5 0.5
##
## Group means:
##           X1           X2
## 0 12.57545 24.85811
## 1 12.41207 24.93275
```

```
# visualise the data
new_data %>%
  add_column(predict(qda_fit, new_data = new_data)) %>%
  ggplot(aes(x=X1,y=X2,fill=.pred_class)) +
  geom_tile() +
  labs(fill = "X3",
       x = "X1",
       y = "X2") +
  theme_bw() +
  viridis::scale_fill_viridis(option="G",
                              begin = 1/6,
                              end = 5/6,
                              discrete=TRUE)
```

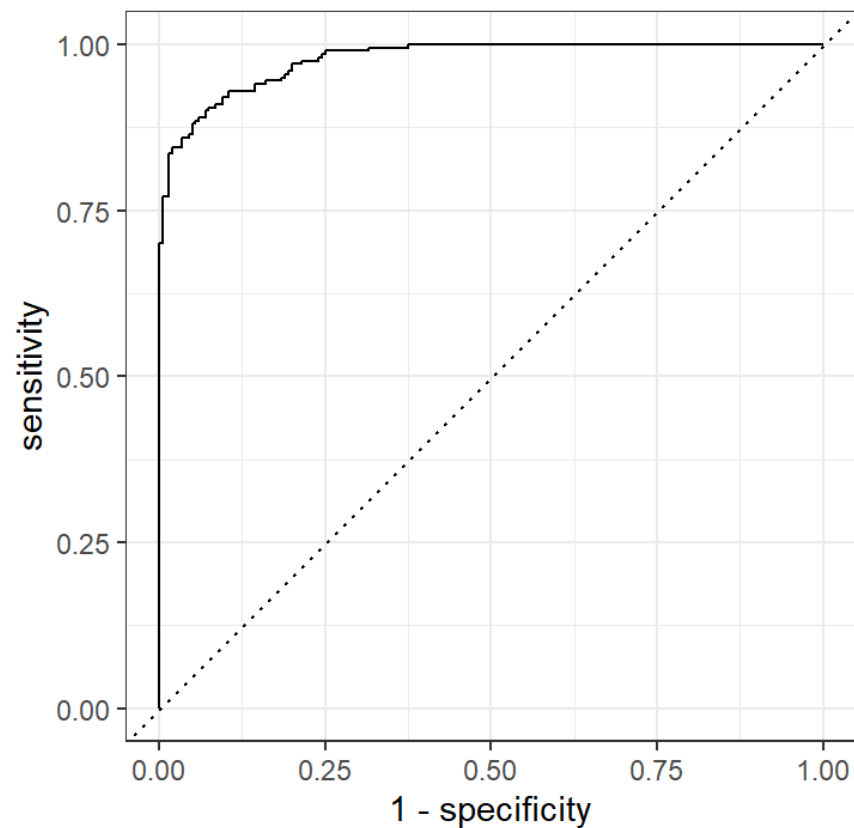


```
# measure performance
qda_pred <- data %>%
  bind_cols(predict(qda_fit, data),
            predict(qda_fit, data, type = "prob"))

qda_pred %>% conf_mat(truth = X3,
                     estimate = .pred_class)
```

```
##           Truth
## Prediction  0   1
##           0 182  18
##           1  18 182
```

```
qda_pred %>% yardstick::roc_curve(truth = X3,
                                  estimate = .pred_0) %>%
  autoplot()
```



Logistic Regression

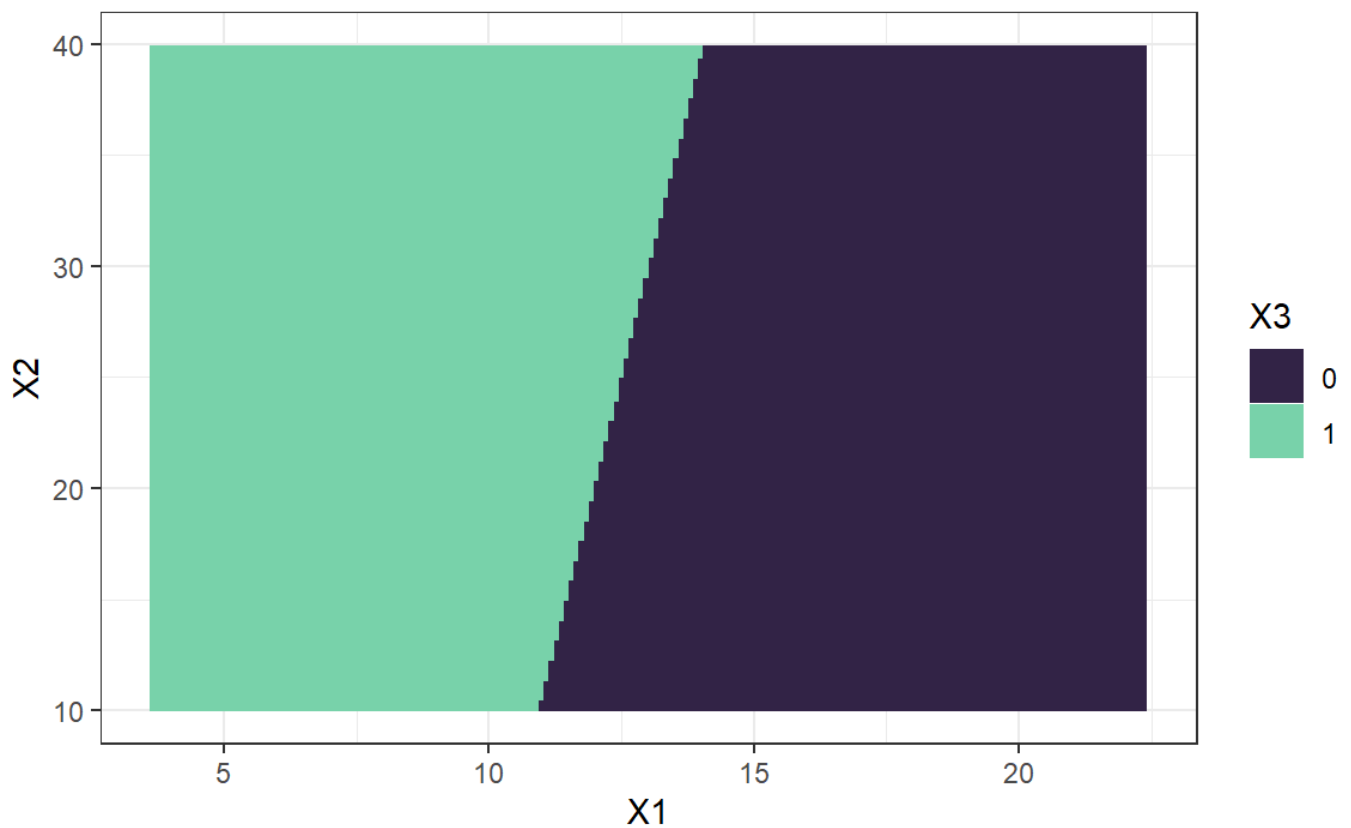
```
# fit the model
lr_r <- recipe(X3 ~ ., data = data)
lr_m <- logistic_reg() %>%
  set_mode("classification") %>%
  set_engine("glm")
lr_wf <- workflow() %>%
  add_recipe(lr_r) %>%
  add_model(lr_m)
lr_fit <- lr_wf %>%
  fit(data)

# model summary
lr_fit %>% extract_fit_parsnip() %>% pluck("fit")
```

```
##
## Call:  stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
##
## Coefficients:
## (Intercept)          X1          X2
##   0.098289   -0.009927    0.001034
##
## Degrees of Freedom: 399 Total (i.e. Null);  397 Residual
## Null Deviance:      554.5
## Residual Deviance: 554.3    AIC: 560.3
```



```
# visualise the data
new_data %>%
  add_column(predict(lr_fit, new_data = new_data)) %>%
  ggplot(aes(x=X1,y=X2,fill=.pred_class)) +
  geom_tile() +
  labs(fill = "X3",
       x = "X1",
       y = "X2") +
  theme_bw() +
  viridis::scale_fill_viridis(option="G",
                              begin = 1/6,
                              end = 5/6,
                              discrete=TRUE)
```

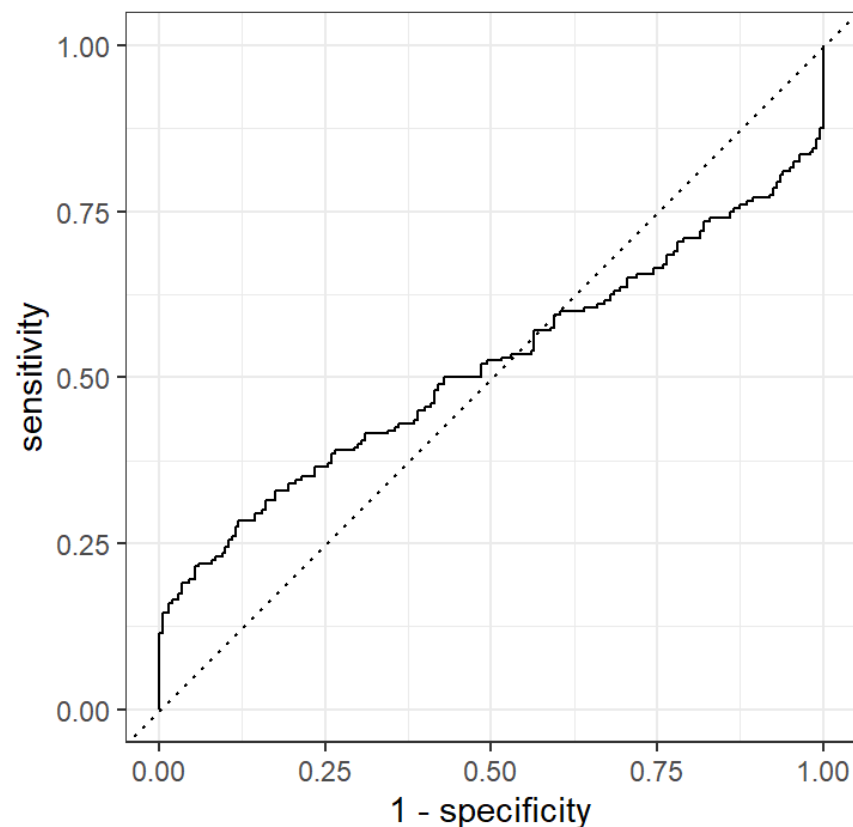


```
# measure performance
lr_pred <- data %>%
  bind_cols(predict(lr_fit, data),
            predict(lr_fit, data, type = "prob"))

lr_pred %>% conf_mat(truth = X3,
                    estimate = .pred_class)
```

```
##           Truth
## Prediction  0   1
##           0 104 98
##           1  96 102
```

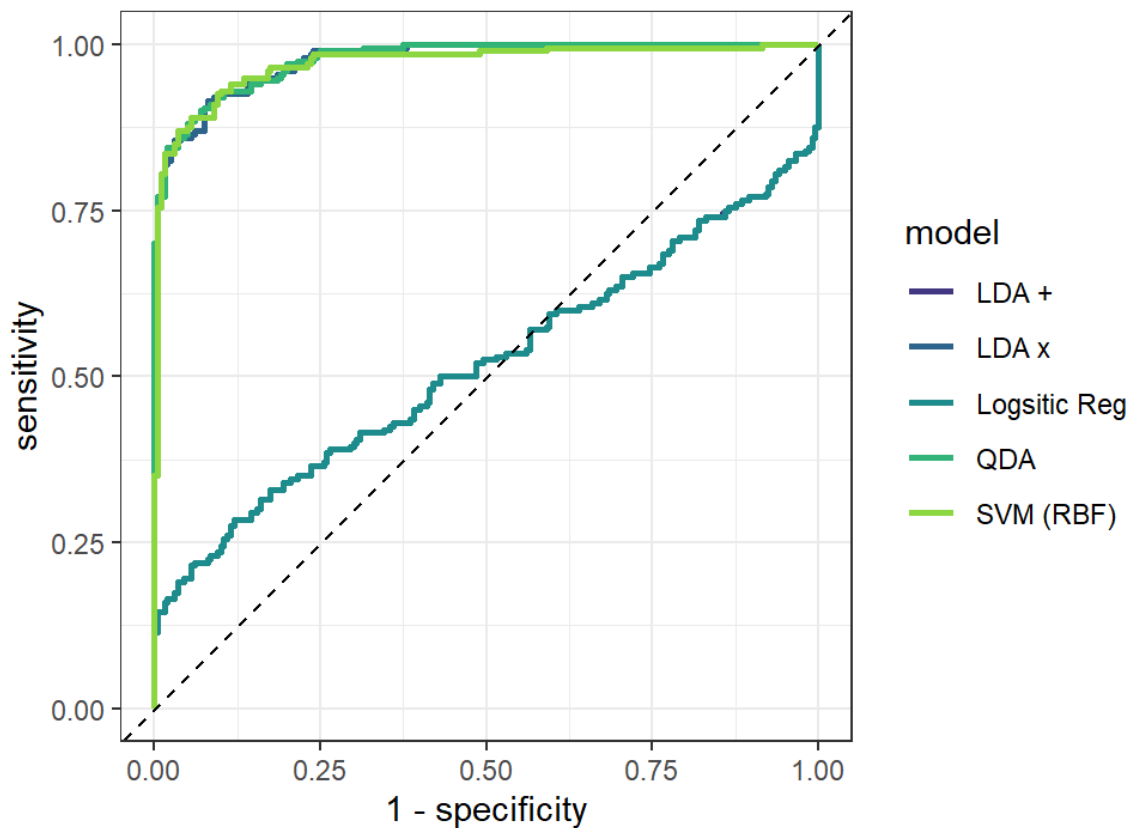
```
lr_pred %>% yardstick::roc_curve(truth = X3,
                                estimate = .pred_0) %>%
  autoplot()
```



Comparing Model Performance

```
# get predictions
preds <- rbind(lda_pred1 %>% add_column(model = "LDA +"),
               lda_pred2 %>% add_column(model = "LDA x"),
               qda_pred %>% add_column(model = "QDA"),
               svm_pred %>% add_column(model = "SVM (RBF)"),
               lr_pred %>% add_column(model = "Logistic Reg"))

# plot ROC curves
preds %>%
  group_by(model) %>%
  yardstick::roc_curve(truth = X3,
                      pred = .pred_0) %>%
  ggplot(aes(x = 1 - specificity, y = sensitivity, color = model)) +
  geom_path(size=1) +
  viridis::scale_color_viridis(option = "D", begin = 1/6, end = 5/6, discrete = TRUE) +
  geom_abline(lty = 2) +
  coord_equal() +
  theme_bw()
```



```
# auc
preds %>%
  group_by(model) %>%
  yardstick::roc_auc(truth = X3,
                     pred = .pred_0)
```

```
## # A tibble: 5 × 4
##   model      .metric .estimator .estimate
##   <chr>      <chr>   <chr>      <dbl>
## 1 LDA +      roc_auc binary      0.516
## 2 LDA x      roc_auc binary      0.977
## 3 Logistic Reg roc_auc binary      0.516
## 4 QDA        roc_auc binary      0.978
## 5 SVM (RBF)  roc_auc binary      0.971
```

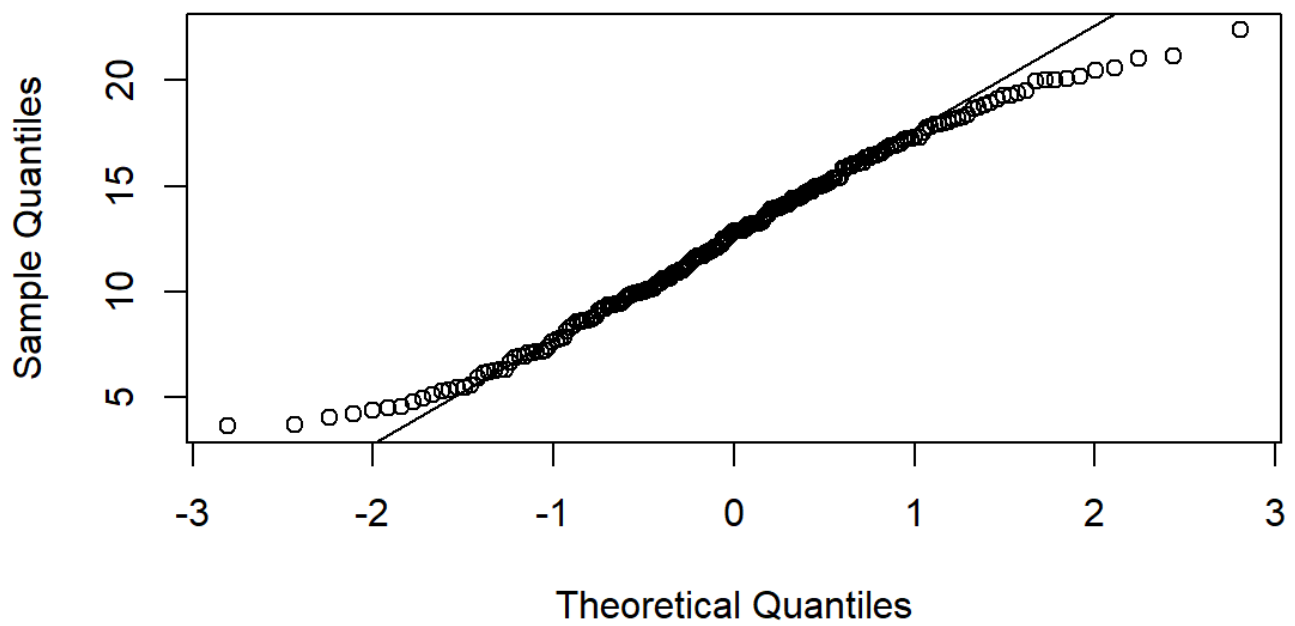
Which Model Should We Choose?

The LDA model without an interaction term is clearly rubbish, as is the logistic regression model. We should not use either of these models. It might be worth fitting a logistic regression model with an interaction term - I suspect it would perform quite well.

```
# assumption checking for LDA/QDA

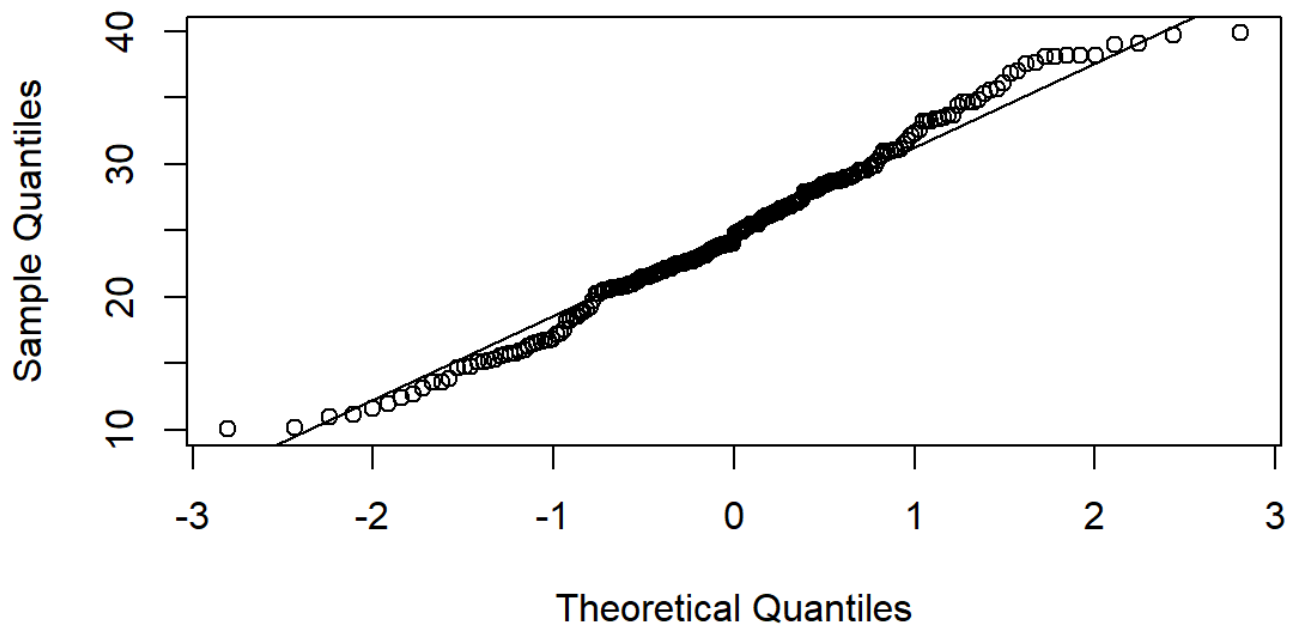
## group 0
### X1 qqplot
qqnorm(data$X1[data$X3==0])
qqline(data$X1[data$X3==0])
```

Normal Q-Q Plot



```
### X2 qqplot  
qqnorm(data$X2[data$X3==0])  
qqline(data$X2[data$X3==0])
```

Normal Q-Q Plot



```
### variances and correlation  
var(data$X1[data$X3==0])
```

```
## [1] 19.34399
```

```
var(data$X2[data$X3==0])
```

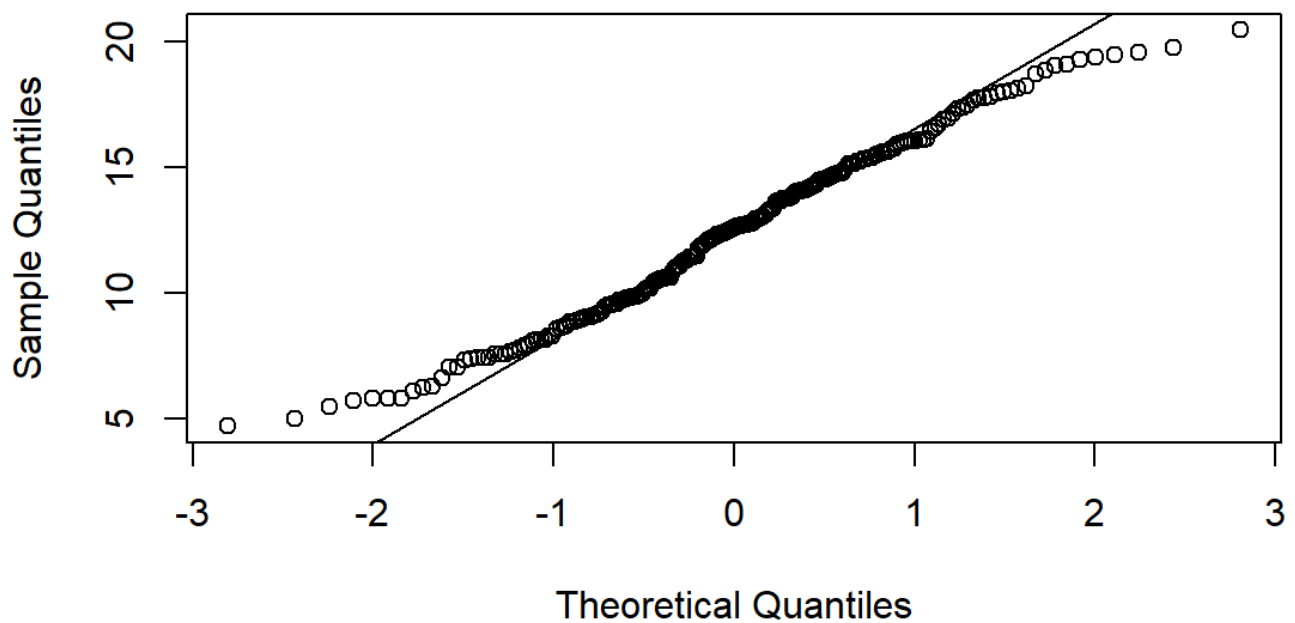
```
## [1] 47.76014
```

```
cor(data$X1[data$X3==0],data$X2[data$X3==0])
```

```
## [1] -0.9628308
```

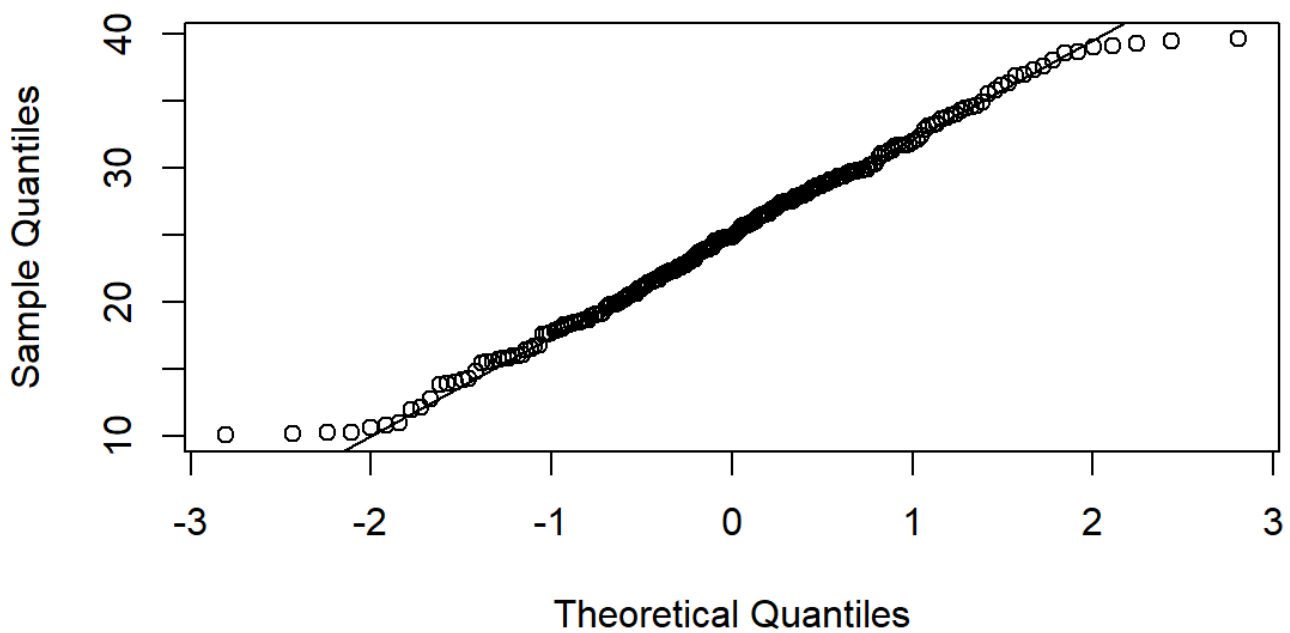
```
## group 1  
### X1 qqplot  
qqnorm(data$X1[data$X3==1])  
qqline(data$X1[data$X3==1])
```

Normal Q-Q Plot



```
### X2 qqplot  
qqnorm(data$X2[data$X3==1])  
qqline(data$X2[data$X3==1])
```

Normal Q-Q Plot



```
### variances and correlation  
var(data$X1[data$X3==1])
```

```
## [1] 13.22029
```

```
var(data$X2[data$X3==1])
```

```
## [1] 49.84151
```

```
cor(data$X1[data$X3==1],data$X2[data$X3==1])
```

```
## [1] 0.9469879
```

LDA assumes multivariate normal for each group - qqplots show this assumption is reasonable. LDA also assumes constant variance/covariance structure for all groups - comparing $\text{cor}(X_1, X_2)$ for each group shows this assumption is unreasonable. Thus, we should not use LDA.

QDA relaxes the assumption of constant variance/covariance structure, so would be a reasonable model to choose in this instance. SVM would also be a reasonable model, though its performance is marginally worse than QDA, and we may want to choose our kernel more carefully.